

GPSS и ПОСП

– примеры лингвистической системы

проблемно-ориентированного языка

(domain-specific language, DSL)

domain-specific language, DSL

(**варианты определения:**

*предметно-ориентированные, предметно-специфические,
проблемно-ориентированные)*

Языки общего назначения:

Универсальные;

Мощные;

Избыточные

Предметно-ориентированные языки:

Узкие;

Ограниченной мощности;

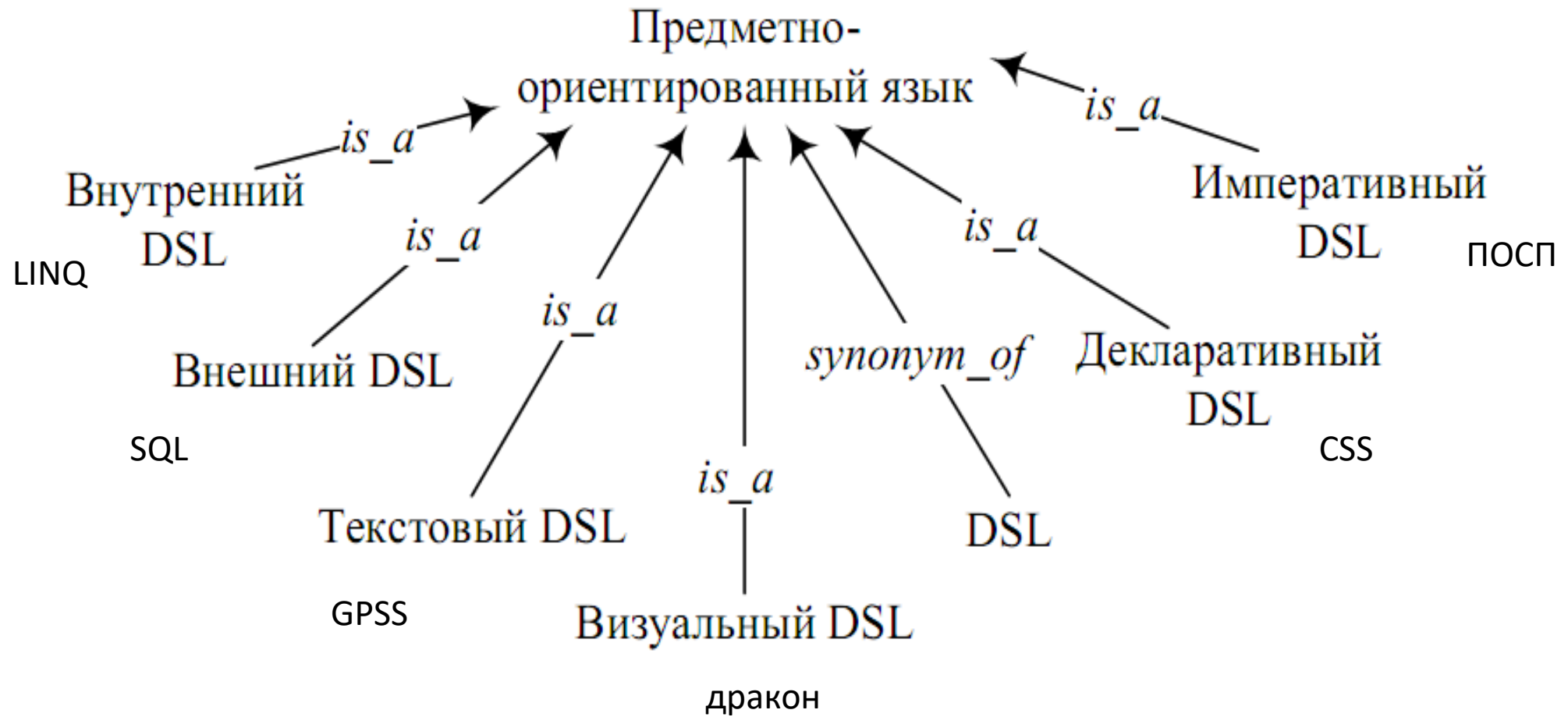
Компактные

“Предметно-ориентированный язык - это язык программирования с ограниченными выразительными возможностями, ориентированный на некую конкретную предметную область” - *Мартин Фаулер*

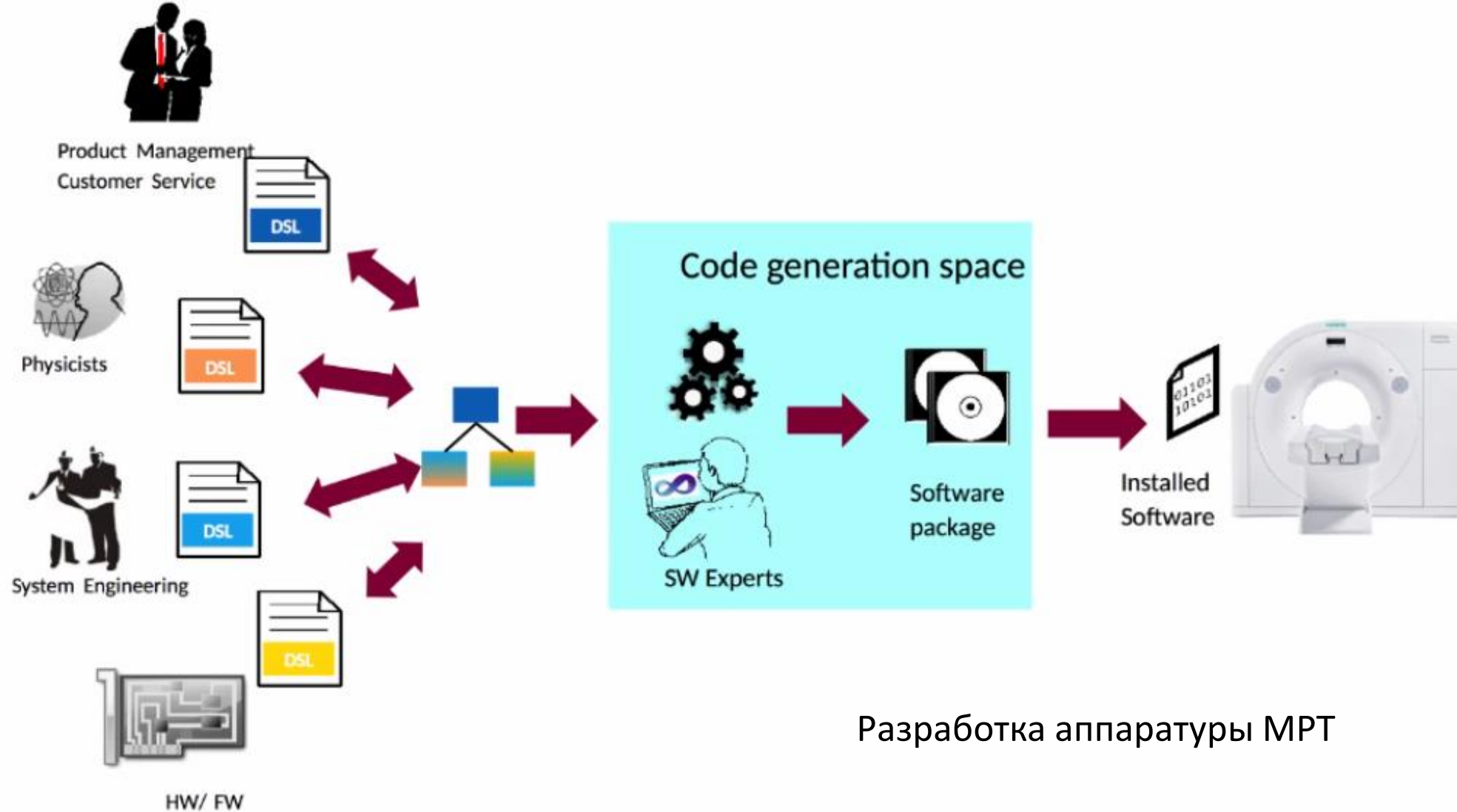
Что такое DSL

- ✓ Язык описания предметной области или определенной проблематики
- ✓ Предназначен для описания решения задачи в терминах и концепциях, приближенных к предметной области
- ✓ Может использоваться приемлемая для конкретной предметной области нотация – от графических схем до описания на естественном языке

Что такое DSL



Что такое DSL



Разработка аппаратуры МРТ

Основная идея спец. языка:

- ✓ предметно/объектно - ориентированный подход
- ✓ компонентное программирование
- ✓ структурный базис для узкоспециализированных задач
- ✓ формализация описания предметной области

основные типы DSL

- внутренний (встраиваемый, internal) DSL;
- внешний DSL (external) — т.е. написан на языке, отличающемся от основного языка программирования.

Есть интегрированные среды разработки

DSL (Language Workbench, MetaCASE)

Встраиваемые DSL / текстовые /

- ✓ Реализуются средствами языка программирования
- ✓ Не требуют специального компилятора
- ✓ Просты в реализации
- ✓ Выразительные средства:
 - ❖ API специальных библиотек (например, .NET)
 - ❖ Объектная модель и интерфейсы работы с ней
 - ❖ Макросы (как в C++)
 - ❖ Специализированные комментарии (типа JavaDoc)
 - ❖ Аннотации (как в C#)
 - ❖ Расширения языка (типа LINQ)

Внешние DSL

текстовые или графические языки

- ✓ Используется полностью новый синтаксис
- ✓ Может иметь стандартизованный формат (XML)
- ✓ Реализует расширение или специализацию UML
- ✓ Необходим специальный транслятор

[но нет ограничений на выразительные средства]

Методология MetaCASE

Продукты MetaCASE представляют собой узкоспециализированные среды разработки приложений, которые создают пользовательский инструментарий на основе высокоуровневого описания требуемых инструментов.

Технология MetaCASE позволяют определять и создавать инструменты CASE, которые поддерживают различные методологии. Средство настройки CASE сначала определяет желаемую методологию и настраивает соответствующий инструментарий CASE, а разработчики ПО используют этот инструмент CASE для разработки программных систем.

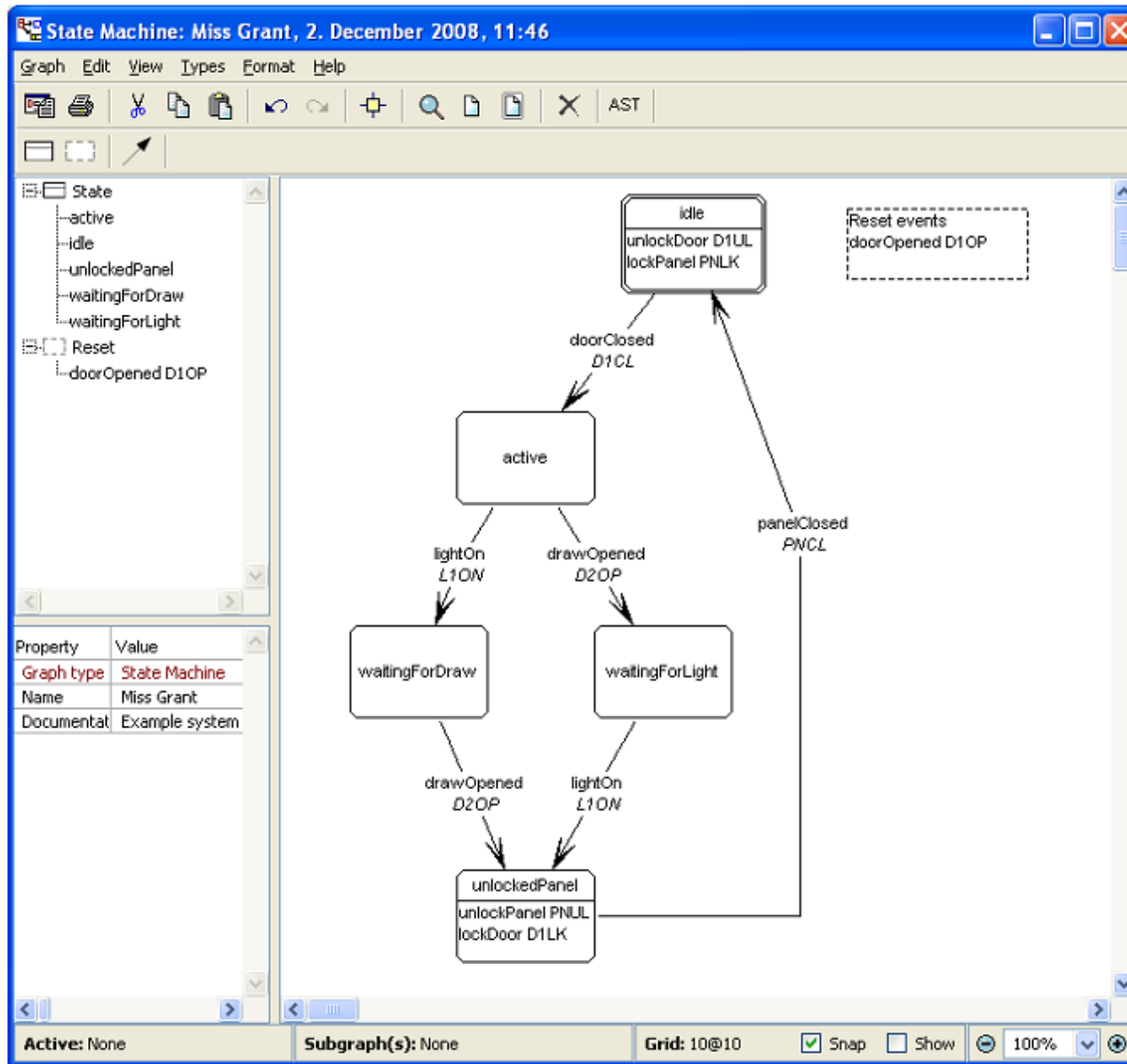
Преимущество этого подхода заключается в том, что один и тот же инструмент используется с разными методологиями, что уменьшает и кривую обучения, и стоимость.

Эту технологию можно использовать в качестве практического инструмента обучения с сокращённой продолжительностью разработки и изучения.

[CASE :: computer-aided software engineering]

*Diagramming Tools; Report Generators; Analysis Tools; Central Repository;
Documentation Generators; Code Generators*

среда разработки DSL (Language workbench MetaEdit)



Примеры metaCASE

DOME

GME

MetaEdit+

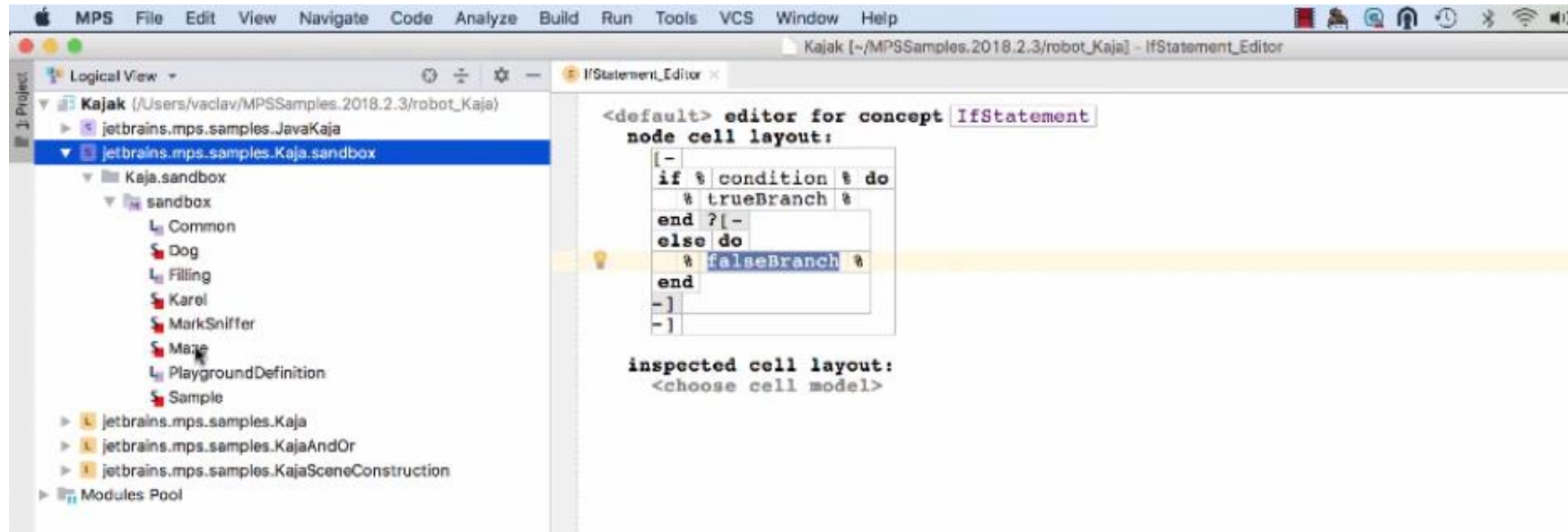
Obeo Designer

Whole Platform

ConceptBase

Jetbrains MPS

среда разработки DSL (Language workbench JetBrains MPS)



С помощью MPS можно самостоятельно создавать редакторы DSL для упрощения использования нового языка.

Специалисты в своих профессиональных областях (не обязательно программисты) смогут работать с этой программной средой, используя язык, созданный на основе специализированной терминологии конкретной предметной области.

Области применения DSL

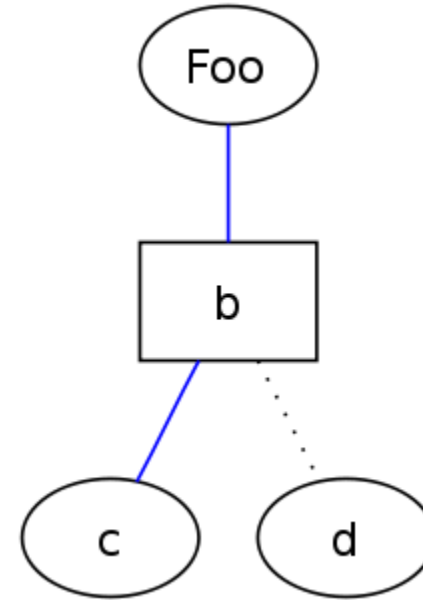
- ❖ Описание преобразования текстов (XSLT, Regular-Expresion)
- ❖ Описание графов (DGML, GraphML, DOT)
- ❖ Описание формул (MathML)
- ❖ Оформление документации (DITA, DocBook, Doxygen, markdown)
- ❖ Описание форматирования текста (TeX)
- ❖ Задание правил для экспертных систем (Drools)
- ❖ Задание спецификаций ПО (ACSL)
- ❖ Описания правил тестирования (JUnit)
- ❖ Model Driven Development (Finite-State-Machine, SCXML)
- ❖ Описание архитектур приложений (AADL)

Области применения DSL

- ❖ Описание разметки данных (HTML, CSS)
- ❖ Описание интерфейсов приложений (XAML, XUL, QML)
- ❖ Описание моделей данных (DDL, ER)
- ❖ Описание доступа к данным в СУБД (SQL, DAX)
- ❖ Описание доступа к структурам данных (XQuery, XPath)
- ❖ Описания сетевых протоколов (SDL)
- ❖ Описания форматов данных (ASN.1)
- ❖ Описания грамматик языков (EBNF, flex, bison)
- ❖ Описания манипуляций над данными (DML)
- ❖ Описание процессов (BPMN, Дракон, ПОСП)

Языки описания графов / DOT, GraphML /

```
graph graphname {  
    // label - видимое название вершины  
    a [label="Foo"];  
    // shape - определение формы вершины  
    b [shape=box];  
    // color - определение цвета ребра  
    a -- b -- c [color=blue];  
    // style - определение стиля ребра  
    b -- d [style=dotted];  
}
```



Язык описания диаграмм / mermaid.js/

flowchart LR

```
svet -. -> a2((сигнал))
```

```
a2 -.-> xsect
```

$$k1((ABTO)) \Rightarrow xsect$$

```
src1 -. -> k1
```

```
xsect ==> ax((X))
```

```
xsect <-.-> k2((перек<br/>ресток))
```

```
subgraph src1
```

```
["K   "]
```

c1(<u>поток</u>)

end

```
subgraph xsect ["Π  "]
```

b(<u>проезд</u>)

end

```
subgraph svet ["A             "]
```

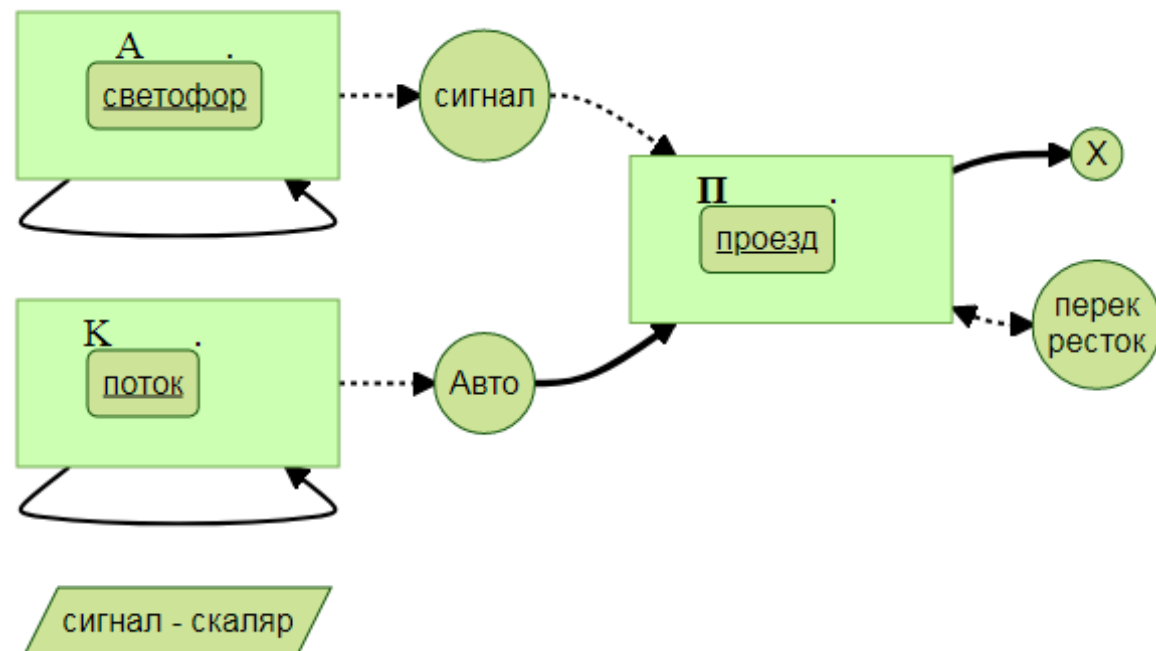
а("<u>светофор</u>")

end

```
src1 --> src1
```

svet --> svet

[/сигнал - скаляр/]



```
classDef blocks width:160px;font-size:20px;font-family:serif;text-align:left;
```

```
class xsect,src1,svet blocks;
```

Язык описания графов /GraphML/

```
<?xml version="1.0" encoding="UTF-8"?>

<graphml xmlns="http://graphml.graphdrawing.org/xmlns"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://graphml.graphdrawing.org/xmlns
http://graphml.graphdrawing.org/xmlns/1.0/graphml.xsd">

  <key id="d0" for="node" attr.name="color" attr.type="string">

    <default>yellow</default>

  </key>

  <key id="d1" for="edge" attr.name="weight" attr.type="double"/>

  <graph id="G" edgedefault="undirected">

    <node id="n0">

      <data key="d0">green</data>

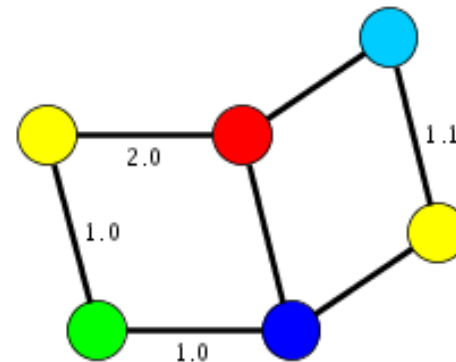
    </node>

    <node id="n1"/>

    <node id="n2">

      <data key="d0">blue</data>

    </node>
```



...

```
<edge id="e0" source="n0" target="n2">
  <data key="d1">1.0</data>
</edge>
```

...

```
</graph>
```


Язык описания алгоритмов / ДРАКОН /

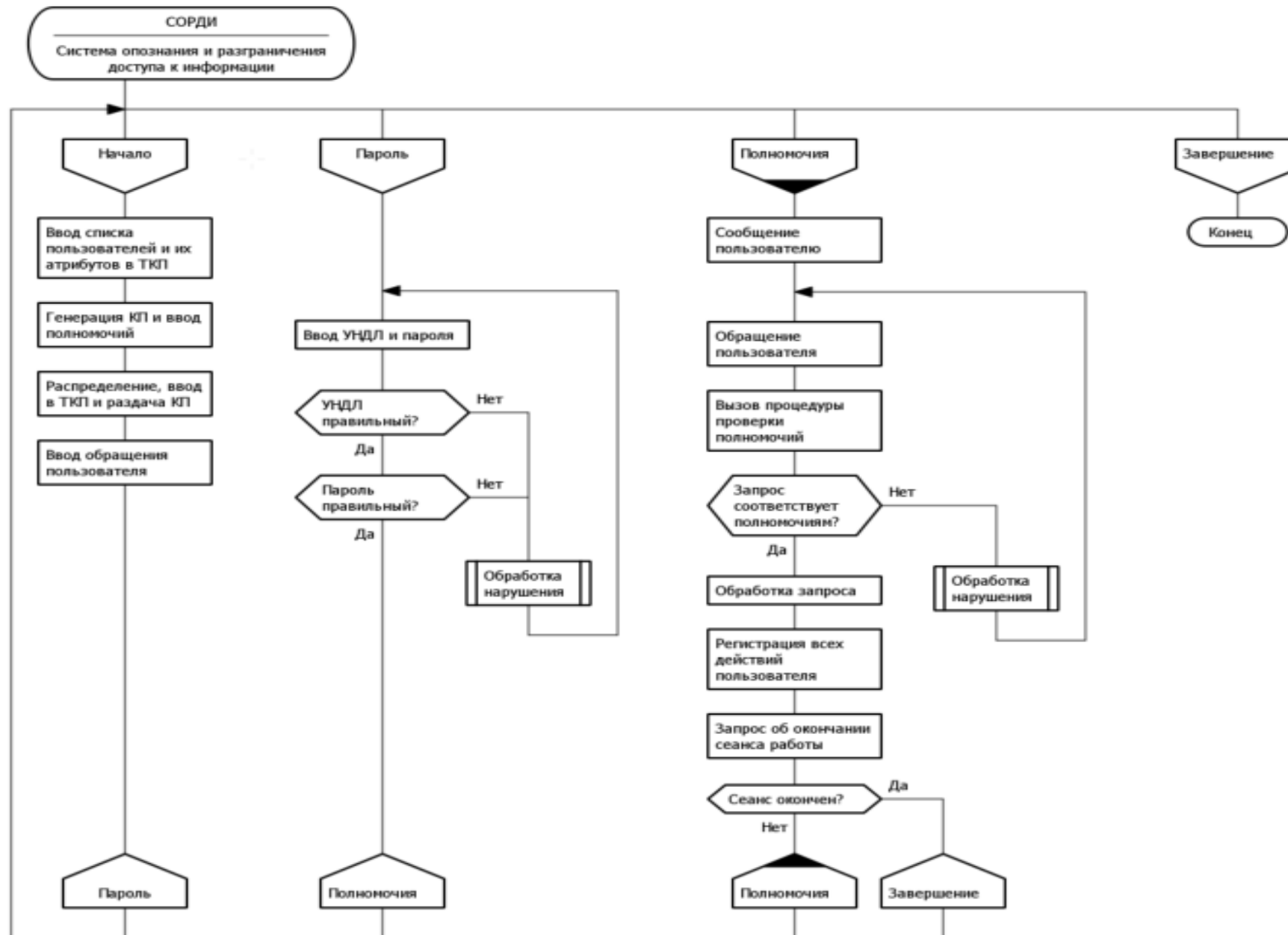
ДРАКОН — это графический алгоритмический язык, созданный в ходе проекта космической системы Буран. Инженеры создавали ДРАКОН, чтобы иметь дело с реальной сложностью систем управления космическими аппаратами. Авторы языка ДРАКОН спроектировали язык таким образом, чтобы снизить когнитивные усилия при работе с алгоритмами. Каждое правило и каждая функциональная возможность ДРАКОНа помогает людям быстрее понимать программы.

Drakon.Tech перенёс язык ДРАКОН в веб-среду, позволяя разработчикам применять эту технологию для веб-приложений.

Процесс разработки в Drakon.Tech включает следующие шаги:

- Разработчик рисует Дракон-схемы в Drakon.Tech IDE.
- Генератор кода создаёт JavaScript-функцию для каждой Дракон-схемы
- Drakon.Tech помещает эти функции в JS-файл, который можно выполнить в браузере или Node.js.

Язык описания алгоритмов / ДРАКОН /



Язык алгоритмов ДРАКОН <https://drakon.tech/>



drakon-tetris > tetris >
buildGame



Проекты

Центр управления

drakon-tetris

html

sworker

tetris

Best score

Events

Game data

Game logic

Install

Landing spot

Render

init

buildGame

gameOver

nextOutline

render

startGame

update

projectmaster

Корзина

buildGame

Только чтение

Window

Format top panel

Game field

Bottom panel

Events

Game state

calculatesizes()

main = get(module.mainId)
main.innerHTML = ""
main.style.fontFamily = "Arial, sans-serif"
main.style.margin = "auto"
main.style.width = calcMainWidth()
main.style.position = "relative"

topPanel = makeTableRow(main)

scoreLabel = makeTableCell(topPanel)
setText(scoreLabel, "Score")
module.scoreDiv = makeTableCell(topPanel)
nextLabel = makeTableCell(topPanel)
setText(nextLabel, "Next")
nextContainer = makeTableCell(topPanel)

module.nextCanvas = createCanvas(
nextContainer,
NextSize,
NextSize
)

Format top panel

module.scoreDiv.style.width = "100%"

topPanel.style.width = "100%"
topPanel.style.background = "black"
topPanel.style.fontSize = "20px"

setTopCellStyle(scoreLabel, "white")
setTopCellStyle(
module.scoreDiv,
"yellow"
)
setTopCellStyle(nextLabel, "white")
module.scoreDiv.style.fontWeight = "bold"

module.nextCanvas.style.background = "black"

Game field

canvasDiv = make(main, "div")

module.canvas = createCanvas(
canvasDiv,
ColumnCount * module.sqSize,
RowCount * module.sqSize
)

module.canvas.style.verticalAlign = "top"

module.ctx = module.canvas.getContext(
"2d"
)

module.ctx.scale(
module.scale,
module.scale
)

Bottom panel

bottomPanel = makeTableRow(main)

bottomPanel.style.width = "100%"
bottomPanel.style.background = "black"
bottomPanel.style.color = "grey"
bottomPanel.style.fontSize = "25px"

makeDownArrow(
bottomPanel,
"fas fa-arrow-left",
onLeft
)

makeDownArrow(
bottomPanel,
"fas fa-redo",
onRotate
)

makeDownArrow(
bottomPanel,
"fas fa-arrow-down",
onDown
)

makeDownArrow(
bottomPanel,
"fas fa-arrow-right",
onRight
)

Events

resetswiper()

hammer = new module.
module.canvas

hammer.get('pinch')
hammer.get('rotate')
hammer.get('pan')
hammer.get('press')
hammer.get('double')
{enable: false
};

hammer.get('swipe')
{ direction:
}

hammer.on("panleft"
hammer.on("panright"
hammer.on("swipeup"
hammer.on("swipedown")

module.canvas.add(
"touchstart",
onTouchStart
)

Последователи GPSS

JGPSS Framework

is discrete time simulation general-purpose open source tool
written in Java

* Modelo de una cafeteria

* Atiende clients

```
clientes Generate Uniform,15,8,0.0,0.0,0.0,0 ;<<--comment-->>
camarero Seize camarero ;<<--comment-->>
          Advance Uniform,4,1 ;<<--comment-->>
          Release camarero ;<<--comment-->>
salida Terminate 1 ;<<--comment-->>
```

Последователи GPSS

go-gpss

is a framework based on concept GPSS (General Purpose Simulation System)

<https://godoc.org/github.com/soldatov-s/go-gpss>

```
```Golang
// Build pipeline
// Generator -> Queue -> Facility -> Hole

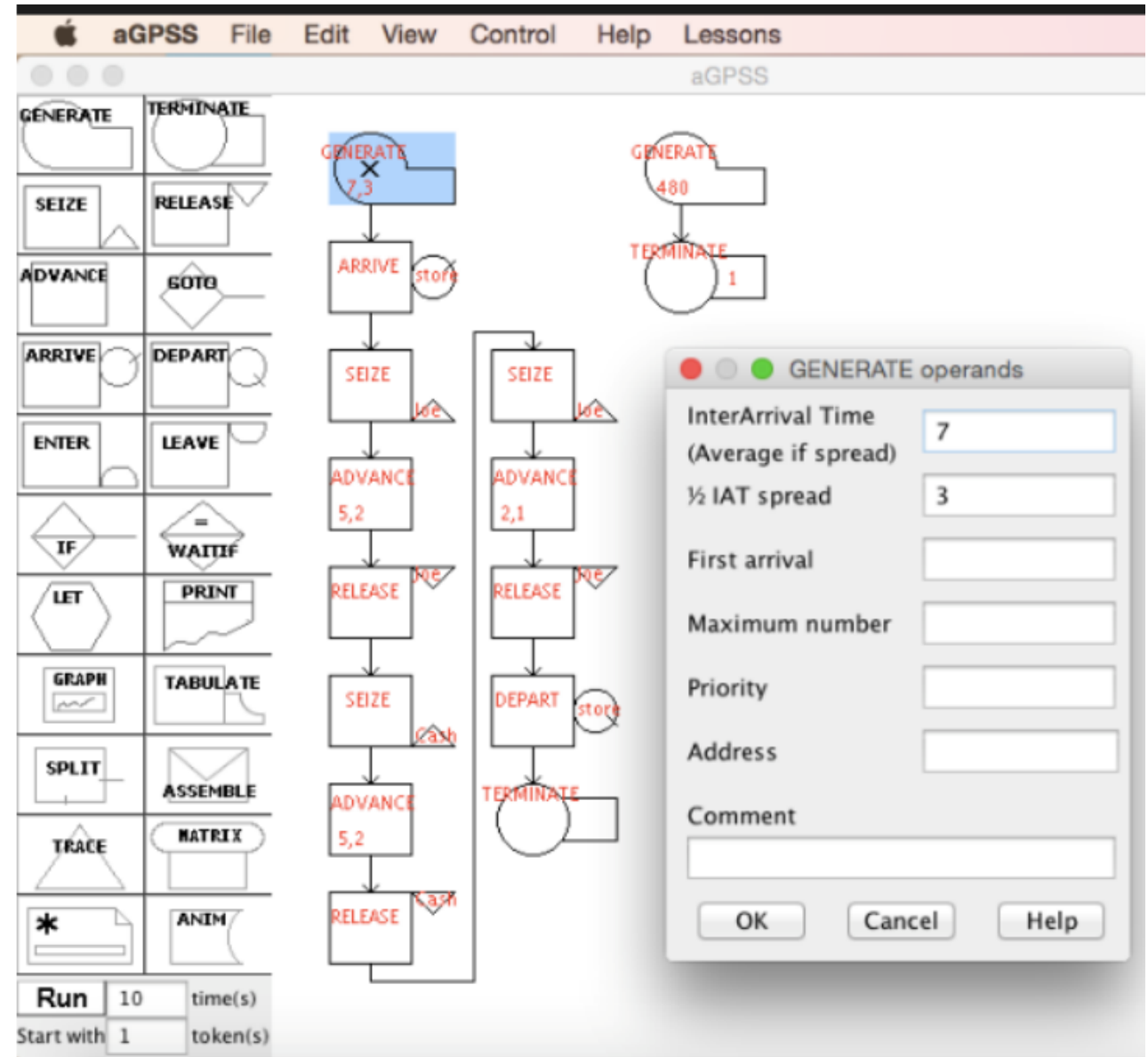
p := objects.NewPipeline("Barbershop")
 AddObject(objects.NewGenerator("Clients", 18, 6, 0, 0, nil))
 AddObject(objects.NewQueue("Chairs"))
 AddObject(objects.NewFacility("Master", 16, 4))
 AddObject(objects.NewHole("Out"))
// Start simulation
p.Start(480)
<-p.Done
p.Report()
```

# Последователи GPSS

## aGPSS

DISCRETE EVENT SIMULATION ON THE  
MACINTOSH FOR BUSINESS STUDENTS

Prof. Ingolf Ståhl  
Department of Economic Statistics  
Stockholm School of Economics



# Моделирование на основе ПОСП

ИОС  
«Процессы»

Проект  
разработки на C#

ИОС Процессы

ФайлПостроениеЗапускСтопОкнаОтображения

serv-disk-fault-

/\* Система состоит из 1 процессора и 1 диска. На систему поступает поток задач. Цикл решения задачи - обработка в процессоре и выгрузка на диск. Для каждой задачи решение в системе индивидуально и распределено равномерно от 3 до 8. У каждой задачи время решения в процессоре в интервале (3,7), время занятия диска - (1,3). В системе одновременно находятся не более 4х задач. Остальные ожидают во внешней очереди. Когда очередная задача выполнит все свои циклы, она покидает систему, и другая из очереди может начать обработку. На диске происходят сбои в интервале (30,50). После сбоя с мгновенным восстановлением, но с разрушением текущих данных, т.е. если сбой произошел во время обработки задачи, то необходима повторная обработка задачи диском. Процессор иногда может быть занят с интервалом 200 на время 10, но без прерывания выполняемой задачи. \*/

блок контроллер

ПОТОК

описание

Цикл- скаляр; // число циклов решения

Тнов- скаляр;

все описание

алгоритм

НАЧ: Цикл := ЦЕЛОЕ(RAND\*5+3);

создать объект W типа вектор (1,2- скаляр);

W(1) := Цикл; // число циклов для обработки

создать объект S типа ссылка;

S := ссылка на W;

Тнов := ВРЕМЯ+RAND\*10+20;

ждать ВРЕМЯ=Тнов;

активизировать инициатор из S на метку СТАРТ блока Хост

направить инициатор на метку НАЧ;

все алгоритм

Модельное время: 110,05

ОбъектыТУТБВГрафикСхема моделиИнициаторы

	Блок	Имя	Значение	Тип
▶	ПОТОК	Цикл	6	Скаляр
	ПОТОК	Тнов	132,982	Скаляр
	Тест	Тстоп	200	Скаляр
	Тест	Ттест		Скаляр
	Сбой	Тсбой	129,188	Скаляр
	Хост	ПРОЦ	"занят"	Скаляр
	Хост	ДИСК	"занят"	Скаляр
	Хост	Макс	3	Скаляр
	Хост	Тдиск	112,4352	Скаляр
	Хост	СбД	"Нет"	Скаляр
	Хост	КСБ	1	Скаляр
	ПОТОК	W	(1, 2)	Вектор
	ПОТОК	S	4	Ссылка

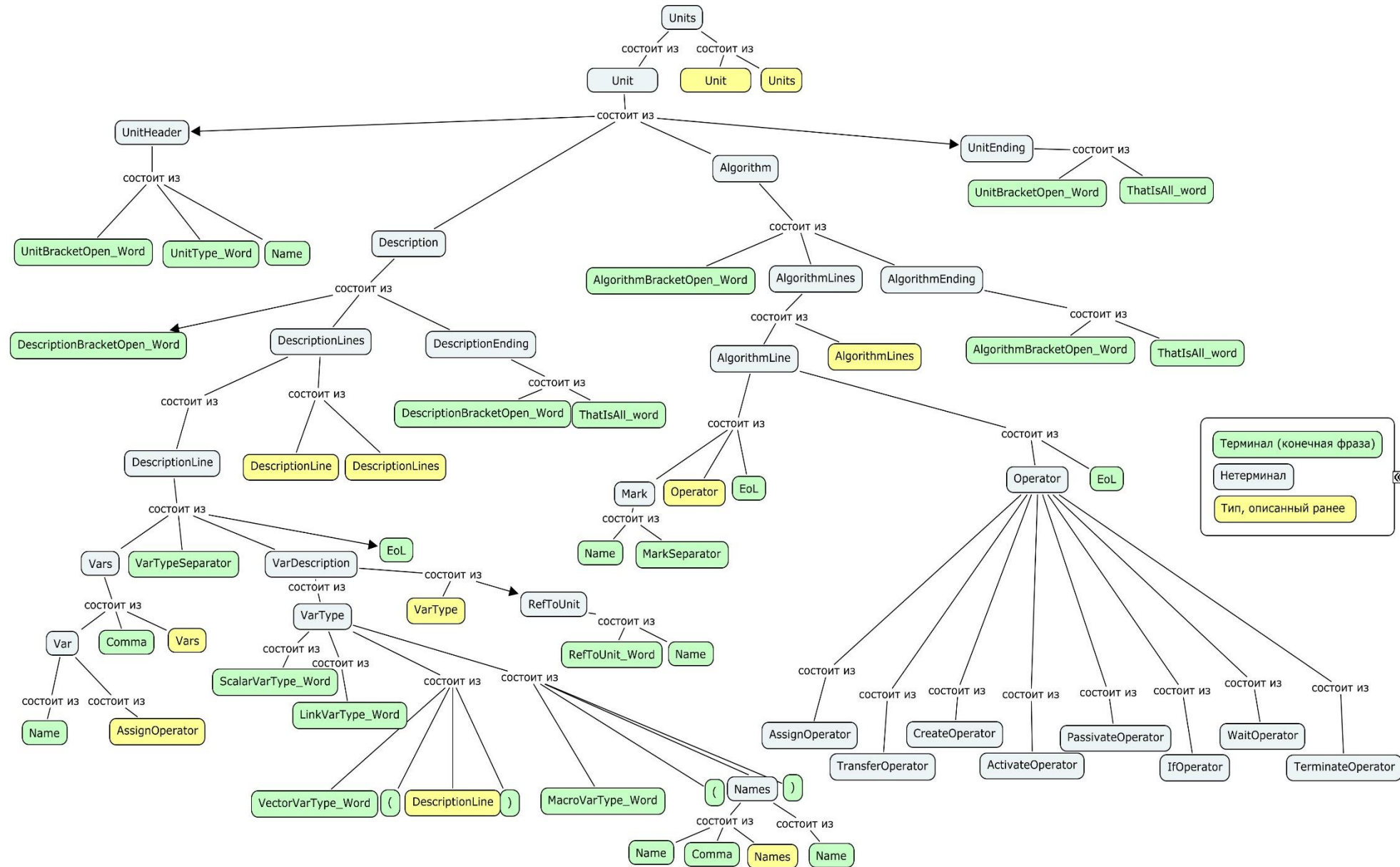
Модель остановлена. Время=:110,0492

# Моделирование на основе ПОСП

ИОС  
«Процессы»

Проект  
разработки на C#

Схема  
синтаксиса  
ПОСП

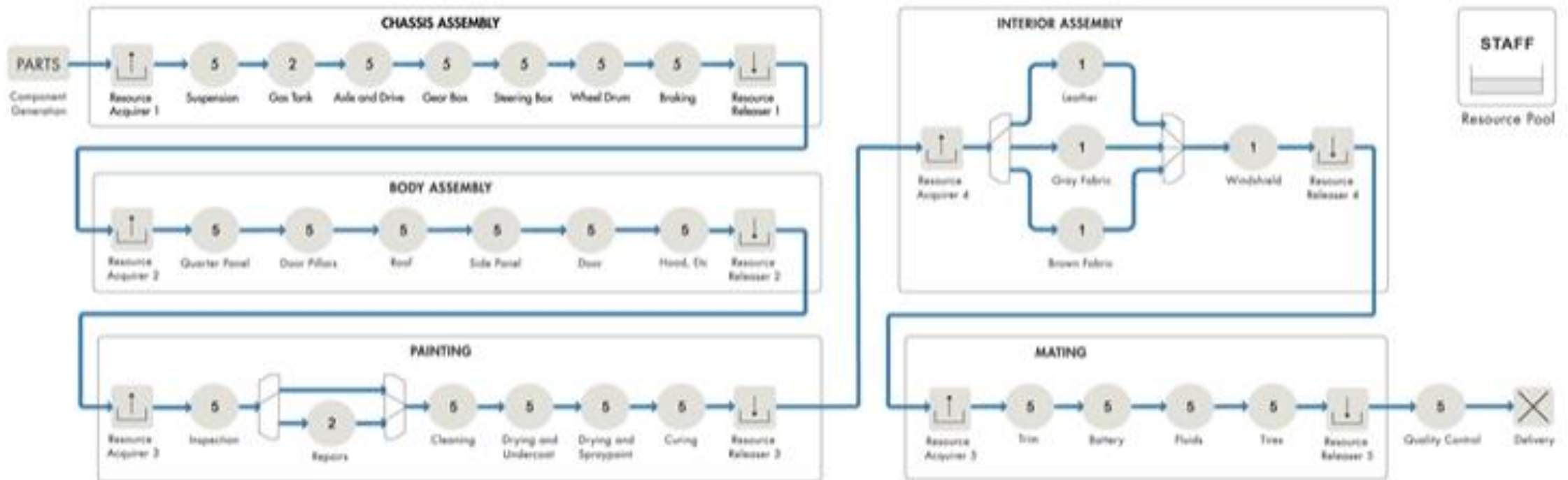




# Варианты систем ИМДП

## MathWorks MatLab

Stateflow предоставляет графический язык, который включает диаграммы перехода состояний, блок-схемы, таблицы перехода состояний и таблицы истинности.



# Варианты систем ИМДП

## MathWorks MatLab

**Stateflow** предоставляет графический язык, который включает диаграммы перехода состояний, блок-схемы, таблицы перехода состояний и таблицы истинности.

Можно использовать **Stateflow**, чтобы описать, как алгоритмы MATLAB и модели Simulink реагируют на события и временные условия.

С помощью **Stateflow** можно проектировать и разрабатывать диспетчерский контроль, планирование задач, управление отказами, протоколы связи, пользовательские интерфейсы и гибридные системы.

**SimEvents** предоставляет механизм моделирования дискретных событий и библиотеку компонентов для анализа моделей систем, управляемых событиями, и оптимизации характеристик производительности (задержка, пропускная способность и потеря пакетов).

С помощью **SimEvents** можно изучать влияние времени выполнения задач и использования ресурсов на производительность вашей системы, проводить оперативные исследования для принятия решений, связанных с прогнозированием, планированием мощностей или управлением цепочкой поставок.

## Варианты систем ИМДП

Проект **SIMODO** предоставляет программные средства математического моделирования для исследования комплексных моделей, захватывающих несколько предметных областей, имеющих длительный цикл производства и поддержки. Важнейшей задачей проекта является возможность создавать структурированные дополняемые модели с использованием специализированной среды моделирования, а также дорабатывать существующие инструменты этой среды и разрабатывать новые, настраивая систему для решения усложняющихся задач.

Языково-ориентированная система моделирования (**ЯСМ**) - технология имитационного математического моделирования, при которой в основе системы лежит **набор** предметно-ориентированных языков (**DSL**), отвечающих за свою предметную область, которую он описывает наиболее полно и просто.

ЯСМ также предполагает наличие средств автоматизации разработки DSL.

The screenshot shows a Qt application window titled "SIMODO - /home/mfetisov/projects/probe/simodo-edit". The window is divided into several sections:

- QDockWidget:** A docked panel on the left titled "Home directory" showing a file explorer view of the project files.
- DocumentWindow:** A central window displaying the content of a file named "README.md".
- TextEditorWidget:** A text editor window on the right showing a build script with stages like "build", "test", "coverage", and "delivery".

Red annotations highlight specific Qt widgets:

- Виджет панели QDockWidget:** Points to the "Home directory" dock panel.
- Виджет панели QDockWidget:** Points to the "README.md" document window.
- Виджет текстового редактора QTextEditWidget:** Points to the build script editor.
- Виджет домашнего каталога HomeDirectoryWidget:** Points to the file explorer view.
- Окно документа DocumentWindow:** Points to the "README.md" window.
- Рамка представления документа DocumentViewFrame:** Points to the frame surrounding the document content.
- Загружаемые процессы Loaded processes:** Points to a terminal window in the background showing the execution of the build script.

## Слои оболочки, плагинов и фоновых процессов как элементы многоуровневой архитектуры

Основой ЯСМ является современный императивный интерпретируемый язык **s-script**, реализующий парадигмы контактного, объектно-ориентированного и функционального программирования (похож на JavaScript).

**s-script** поддерживает модульную структуру, векторную и матричную арифметики, многопоточность и распределённость вычислений, дженерики и другие возможности. Интерпретатор языка использует динамическую компиляцию кода (Just-in-Time).

На основе базового языка s-script построены другие языки ЯСМ:

- язык описания систем обыкновенных дифференциальных уравнений **s-ode**;
- язык логического вывода **s-logic**.

ЯСМ разрабатывалась как инструмент, который можно адаптировать для конкретной сложной модели, имея возможность использовать, как поставляемые языки и компоненты, так и дорабатывая компоненты и плагины на языке **C++**.

Сайт проекта – <http://simodo.ru/>

Работу с **SIMODO** можно рассматривать с точки зрения двух подходов:

- В концепции универсального языка (например, записывая на нём сценарии веб-сервисов), добавляя в него при необходимости импорт модулей на предметно-ориентированных языках (например, для описания логического вывода или дифференциальных уравнений).
- В концепции классического моделирования, записывая модели на предметно-ориентированных языках, а универсальный язык использовать только для описания сценария моделирования.

Такая дуальность **SIMODO** обеспечивается за счёт современного языка сценариев **s-script**, который не имеет явного уклона в сторону моделирования, и набора предметно-ориентированных языков, на которых пишутся отдельные модули.

Проект SIMODO открыт по адресу <https://gitverse.ru/simodo/loom>

## Методология создания DSL

**Анализ:** 1) определение спектра задач предметной области;  
2) сбор информации о предметной области;  
3) объединение знаний в семантические нотации и операции над ними;  
4) проектирование DSL для лаконичного описания предметной области.

**Реализация:** 1) конструирование библиотеки семантических нотаций;  
2) проектирование и реализация транслятора/транспилятора DSL-программ

**Использование:** 1) написание DSL программ для возможных приложений;  
2) компиляция программ.

## Внешние DSL : шаги разработки

- Определение цели создания языка описания предметной области
- Выбор средств представления языка описания предметной области
- Разработка элементов языка описания предметной области
- Реализация языка описания предметной области

инструментальными средствами



## Инструментарий разработки DSL

### */ Описание грамматик языков /*

- ✓ форма Бэкуса-Наура (BNF)
- ✓ расширенная форма Бэкуса-Наура (EBNF)
- ✓ JetBrains MPS (meta programming system)
- ✓ Lex + YACC
- ✓ JavaCC grammar file
- ✓ ANTLR Grammar
- ✓ Microsoft Visual Studio Modeling SDK (DSL Toolkit)

## Описание грамматик языков в форме Бэкуса-Наура (EBNF)

<цифра> ::= 0-9

<число> ::= {<цифра>}+

<вырж> ::= <терм>[(\+|\-)<вырж>]

<терм> ::= <множ>[(\*/)<терм>]

<множ> ::= [<множ>^]<степ>

<степ> ::= \( <вырж> \) | <идентификатор> | <число>

<программа> ⇒ <вырж>

⇒ <вырж>

⇒ <идентификатор> = <множ>

⇒ a = <множ>

⇒ a = <терм> + <терм>

⇒ a = <идентификатор> + <терм>

⇒ a = b + <терм>

⇒ a = b + <константа>

Формальная система определения синтаксиса, в которой одни синтаксические категории последовательно определяются через другие. Используется для описания контекстно-свободных формальных грамматик. Предложен **Никлаусом Виртом**. Является расширенной переработкой **форм Бэкуса-Наура** и отличается более «ёмкими» конструкциями, позволяющими при той же выразительной способности упростить и сократить описание.

<https://dic.academic.ru/dic.nsf/ruwiki/493710>

ISO / IEC 14977

## Основные преимущества DSL

- ✓ DSL позволяет выражать решения в форме идиом и на высоком уровне абстракций
- ✓ DSL программы лаконичны, самодокументированы для больших объемов, могут быть повторно использованы
- ✓ Использование DSL увеличивает продуктивность, надежность, удобство сопровождения и переносимость
- ✓ DSL включают в себе предметные знания, что делает возможным их сохранение и повторное использование
- ✓ DSL разрешают валидацию данных и оптимизацию на уровне предметной области
- ✓ Использование DSL улучшает тестируемость программ

# ИСТОЧНИКИ

<http://martinfowler.com>

Jetbrains Meta Programming System

[https://www.jetbrains.com/ru-ru/  
mps/concepts/domain-specific-languages/](https://www.jetbrains.com/ru-ru/mps/concepts/domain-specific-languages/)

[https://ru.wikipedia.org/wiki/Предметно-ориентированный\\_язык](https://ru.wikipedia.org/wiki/Предметно-ориентированный_язык)

